H1N1 Outbreak Model

11-compartment ODE system with a 9-day infectious period

Final course project presentation

Goal: build, calibrate, and interpret a stage-structured epidemic model that matches the condition $I(7) = 10$ in a closed population of 1500.

Prepared from the final written report and simulation results

Closed population

9-day infection chain

Numerical calibration

Key inputs

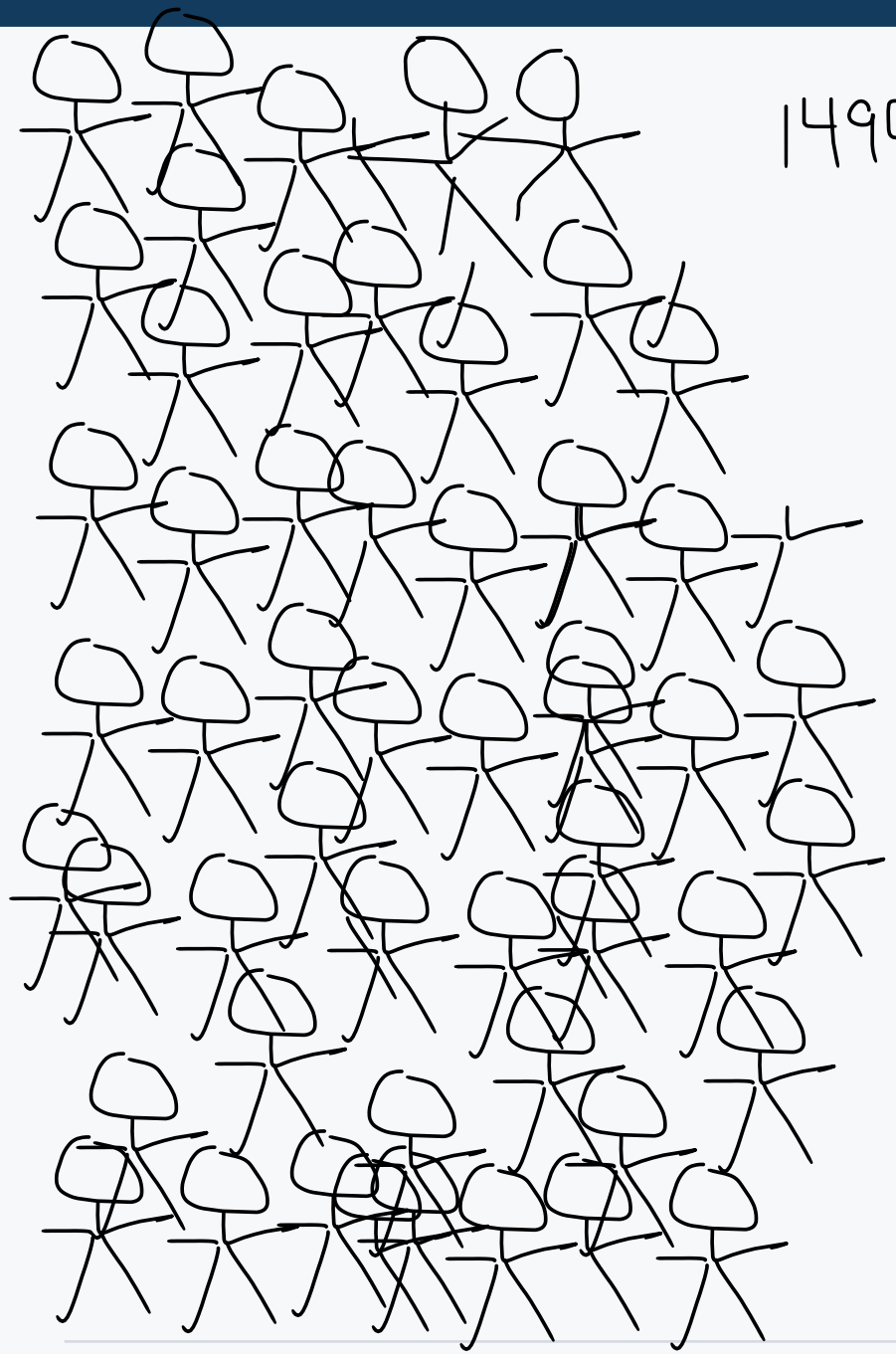
$N = 1500$
$S(0) = 1499$
$I_1(0) = 1$
$R(0) = 0$

Problem

What the project asks us

5.10.3 The H1N1 Virus

The H1N1 virus, also known as the “bird flu,” is a particularly virulent bug but thankfully is also very predicable. Once a person is infected they are infectious for 9 days. Assume that a closed population of $N = 1500$ people (like a small college campus) starts with exactly 1 infected person and hence the remainder of the population is considered susceptible to the virus. Furthermore, once a person is recovered they have an immunity that typically lasts longer than the outbreak. Mathematically we can model an H1N1 outbreak of this kind using 11 compartments: susceptible people (S), 9 groups of infected people (I_j for $j = 1, 2, \cdots, 9$), and recovered people (R). Write and numerically solve a system of 11 differential equations modeling the H1N1 outbreak assuming that susceptible people become infected at a rate proportional to the product of the number of susceptible people and the total number of infected people. You may assume that the initial infected person is on the first day of their infection and determine and unknown parameters using the fact that 1 week after the infection starts there are 10 total people infected.



1499 susceptible

1 infected

Problem statement

What the project asks us to determine

- Model the spread of an H1N1 virus in a closed population of 1500 people.
- Track infection across 9 daily infectious stages instead of one single infected class.
- Determine the unknown transmission parameter β.
- Numerically solve the 11 coupled differential equations.
- Analyze outbreak size, timing of the peak, and the final susceptible/recovered counts.

Given data

Population size

1500

Infectious period

9 days

Calibration target

I(7) = 10

Initial condition

1499 susceptible
1 infected
0 recovered

Math Background: What is a differential Equation

What is a differential equation and why should it be used?

Examples

$$F = ma \text{ or } m\frac{d^2x}{dt^2} = F(x,t))$$

$$\frac{dN}{dt} = rN$$

What is a differential equation

- A mathematical equations that relate an unknown function to its derivatives, representing how a quantity changes over time or space
    - Ordinary Differential Equations (ODE): derivative wrt single independent variable
    - Partial Differential Equations (PDE)

Mathematical background and model structure

Why an 11-compartment system is used

$$S(t) \longrightarrow I_1(t) \longrightarrow I_2(t) \longrightarrow \cdots \longrightarrow I_9(t) \longrightarrow R(t)$$

Core definitions

$$I(t) = \sum_{j=1}^{9} I_j(t)$$

Total infected population

$$\text{infection rate} = \frac{\beta}{N} S(t) I(t)$$

Mass-action incidence

$$\gamma = 1 \text{ day}^{-1}$$

One day spent in each infected stage

Why stage the infected class?

- A standard SIR model uses one infected compartment; this project requires an explicit 9-day infectious period.
- Splitting infection into I_1 through I_9 tracks how long each person has been infected.
- The structure matches the biology: individuals recover only after completing day 9 of infection.

Model equations with parameter substitution

Calibrated transmission parameter: $\beta \approx 0.339976$ day^{-1}

$$\frac{dS}{dt} = -\frac{\beta}{1500} S \sum_{j=1}^{9} I_j$$

$$\frac{dI_1}{dt} = \frac{\beta}{1500} S \sum_{j=1}^{9} I_j - I_1$$

$$\frac{dI_j}{dt} = I_{j-1} - I_j, \qquad j = 2, \ldots, 9$$

$$\frac{dR}{dt} = I_9$$

Initial conditions

$$S(0) = 1499, \quad I_1(0) = 1, \quad I_2(0) = \cdots = I_9(0) = 0, \quad R(0) = 0$$

Population is conserved: $S(t) + \sum I_j(t) + R(t) = 1500$ for all t.

Numerical substitution

β / N

0.339976 / 1500 = 0.0002266507

Initial total infected

I(0) = 1

Initial infection rate

≈ 0.33975

So at t = 0:

dS/dt ≈ -0.33975

dI_1/dt ≈ -0.66025

Math Background: Alternatives to SIR

What is a differential equation and why should it be used?

Alternative Methods

SEIR: Susceptible Exposed Infected Recovered

- Add a series of "Exposed" compartments before the "Infected" compartments

ABM: Agent Based-Modeling

- Would simulate actual contact networks rather than assuming random mixing

Problems with SIR

- Biological and Epidemiological Limitations
- Structural Limitations
- Mathematical Limitations

Why we used SIR?

- Simplicity
- Alternatives are significantly more computationally expensive
- Question explicitly asks us for 11 compartmental differential equations (SIR)

Runge-Kutta method

Numerical background behind the ODE solver

Classical RK45 formulas

$$\mathbf{y}' = \mathbf{f}(t, \mathbf{y}), \qquad \mathbf{y}(t_n) = \mathbf{y}_n$$

$$\begin{aligned}
k_1 &= \mathbf{f}(t_n, \mathbf{y}_n) \\
k_2 &= \mathbf{f}\left(t_n + \frac{h}{2}, \mathbf{y}_n + \frac{h}{2}k_1\right) \\
k_3 &= \mathbf{f}\left(t_n + \frac{h}{2}, \mathbf{y}_n + \frac{h}{2}k_2\right) \\
k_4 &= \mathbf{f}(t_n + h, \mathbf{y}_n + hk_3) \\
\mathbf{y}_{n+1} &= \mathbf{y}_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)
\end{aligned}$$

The method combines four slope estimates over one step of size h.

Why it is useful here

- It is designed for nonlinear ODE systems, which matches our coupled epidemic model.
- Instead of using only one slope like Euler's method, it averages multiple slopes within each step.
- We apply it to the full state vector $Y(t) = (S, I_1, \ldots, I_9, R)^T$.

Project-specific note

In Python we used solve_ivp(method="RK45"), an adaptive Runge-Kutta method from the same family.

Model parameters and initial conditions

Known quantities before solving the system (NumPy)

1 Set parameters and y0 → 2 Define the ODE system → 3 Solve with RK45 → 4 Fit β with Brent's method

Python setup

```
N = 1500                    # total population
N_STAGES = 9                # infected stages I1,...,I9
GAMMA = 1.0                 # progression rate: 1 stage per day
TARGET_I7 = 10.0            # calibration condition: I(7) = 10
T_END = 200                 # simulation length (days)
BETA_BRACKET = (0.3, 0.4) # search interval for beta

y0 = np.zeros(N_STAGES + 2)
y0[0] = N - 1               # S(0) = 1499
y0[1] = 1                   # I1(0) = 1
# I2...I9 and R start at 0
```

What this sets up

- • N = 1500 defines the closed population.
- • N_STAGES = 9 gives the infected stages I_1 through I_9.
- • GAMMA = 1 means people move one stage per day.
- • TARGET_I7 = 10 is the week-1 calibration condition.
- • y0 stores the initial state: 1499 susceptible, 1 infected, 0 recovered.

total_infected_at_day_7(β)−10=0

$$S(t) \longrightarrow I_1(t) \longrightarrow I_2(t) \longrightarrow \cdots \longrightarrow I_9(t) \longrightarrow R(t)$$

Defining the 11-compartment ODE system

Python function for the model equations (NumPy)

1 Set parameters and y0 → 2 Define the ODE system → 3 Solve with RK45 → 4 Fit β with Brent's method

ODE Setup

```
def h1n1_system(t, y, beta):
    S = y[0]
    I = y[1:1 + N_STAGES]
    Itot = np.sum(I)

    dydt = np.zeros_like(y)

    dydt[0] = -(beta / N) * S * Itot
    dydt[1] = (beta / N) * S * Itot - GAMMA * I[0]

    for j in range(1, N_STAGES):
        dydt[j + 1] = GAMMA * I[j - 1] - GAMMA * I[j]

    dydt[-1] = GAMMA * I[-1]
    return dydt
```

Meaning of each part

- S is the susceptible population.
- I stores the 9 infected compartments.
- Itot = sum(I) is the total infected population.
- dydt[0] models susceptibles becoming infected.
- dydt[1] places new infections into I_1.
- The loop moves people through I_2 to I_9.
- dydt[-1] moves people from I_9 into recovery.

$$S(t) \longrightarrow I_1(t) \longrightarrow I_2(t) \longrightarrow \cdots \longrightarrow I_9(t) \longrightarrow R(t)$$

Why we use Runge-Kutta (RK45)

What it is and why it fits this project

What RK45 does

A numerical method for approximating solutions to ordinary differential equations.

Starts from the initial condition vector y0 and repeatedly estimates slopes to move the solution forward in time.

RK45 is adaptive, so it automatically adjusts the step size for accuracy and efficiency.

Why we use it here

Our outbreak model is nonlinear because the infection term contains $S \cdot I$.

That makes the full 11-equation system difficult to solve analytically by hand.

solve_ivp(method="RK45") returns numerical values for $S(t)$, $I_1(t)$, ..., $I_9(t)$, and $R(t)$.

Numerically solving the model

Runge-Kutta simulation with solve_ivp

1 Set parameters and y0 → 2 Define the ODE system → 3 Solve with RK45 → 4 Fit β with Brent's method

solve_ivp with RK45

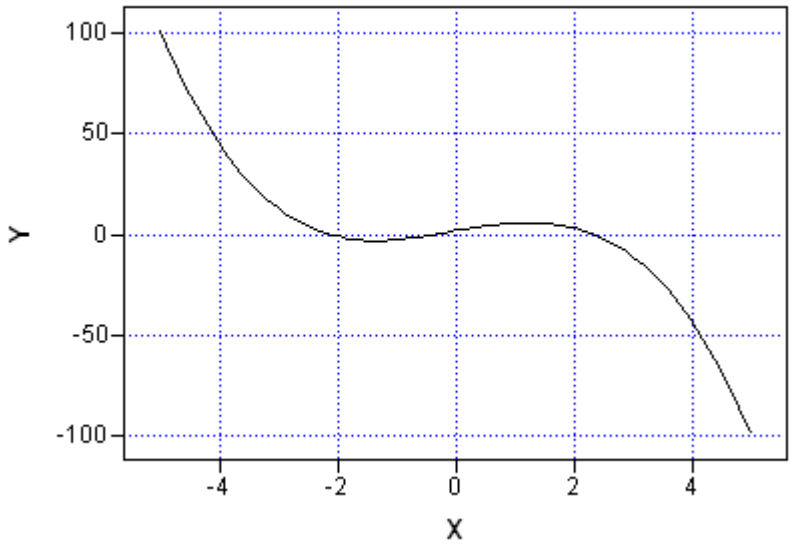
```
def solve_model(beta, t_end=T_END, num_points=2001):
    t_eval = np.linspace(0, t_end, num_points)

    sol = solve_ivp(
        fun=lambda t, y: h1n1_system(t, y, beta),
        t_span=(0, t_end),
        y0=y0,
        t_eval=t_eval,
        method="RK45",
        rtol=1e-9,
        atol=1e-12
    )

    if not sol.success:
        raise RuntimeError(f"ODE solver failed: {sol.message}")

    return sol
```

Why this step is needed

- The system is nonlinear because of the term S · I.
- solve_ivp computes numerical values for all 11 compartments.
- method="RK45" uses an adaptive Runge-Kutta solver.
- t_eval stores the solution at 2001 time points.
- This produces the full outbreak curves over 200 days.

Why we use Brent's method

Root finding for the unknown transmission parameter β

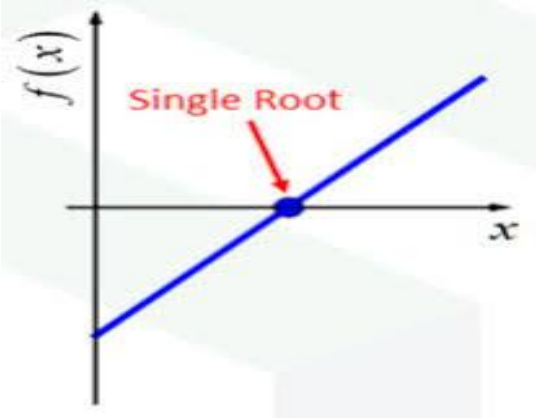


What Brent's method does

A root-finding algorithm that searches for a value where an objective function equals zero.

In our code, the objective is $f(\beta) = I(7;\beta) - 10$.

The search is restricted to a bracket, here (0.3, 0.4), where the solution is expected to lie.



Why we use it here

β is unknown, so we cannot run the final outbreak simulation until it is calibrated.

For each trial β, we solve the ODE system to day 7 and compute $I(7) = I_1(7)+\cdots+I_9(7)$.

If I(7) is above or below 10, Brent's method keeps refining β until $I(7)(\beta) - 10 = 0$.

Brent's method was a good choice because it is both robust and efficient for a one-variable root-finding problem like ours

Key idea: Brent's method is how we determine the calibrated transmission rate β ≈ 0.339976.

Calibrating the transmission parameter β

Using the condition I(7) = 10

1 Set parameters and y0 → 2 Define the ODE system → 3 Solve with RK45 → 4 Fit β with Brent's method

Finding β with brentq

```
def total_infected_at_day_7(beta):
    sol = solve_ivp(
        fun=lambda t, y: h1n1_system(t, y, beta),
        t_span=(0, 7),
        y0=y0,
        t_eval=[7],
        method="RK45",
        rtol=1e-9,
        atol=1e-12
    )
    infected_stages = sol.y[1:1 + N_STAGES, 0]
    return np.sum(infected_stages)

def fit_beta(target_i7=TARGET_I7, bracket=BETA_BRACKET):
    def objective(beta):
        return total_infected_at_day_7(beta) - target_i7

    return brentq(objective, bracket[0], bracket[1])
```

What this does

- β is unknown before solving the model.
- For each trial β, we solve the system up to day 7.
- We compute $I(7) = I_1(7) + \cdots + I_9(7)$.
- The objective function is $I(7) - 10$.
- brentq searches the interval (0.3, 0.4) for the β value that makes the objective equal 0.

Results overview

Calibrated model outputs

Key numerical results

Transmission parameter β	**0.339976**
Peak total infected	**787.77 people**
Time of peak infection	**27.1 days**
Final susceptible	**83.32 people**
Final recovered	**1416.68 people**

Main takeaway

The outbreak infects most of the population before it dies out.

- A relatively large calibrated β creates a fast epidemic wave.
- The peak occurs around day 27, when the infected population is largest.
- Only about 83 people remain uninfected at the end of the simulation.

Graph 1 Code

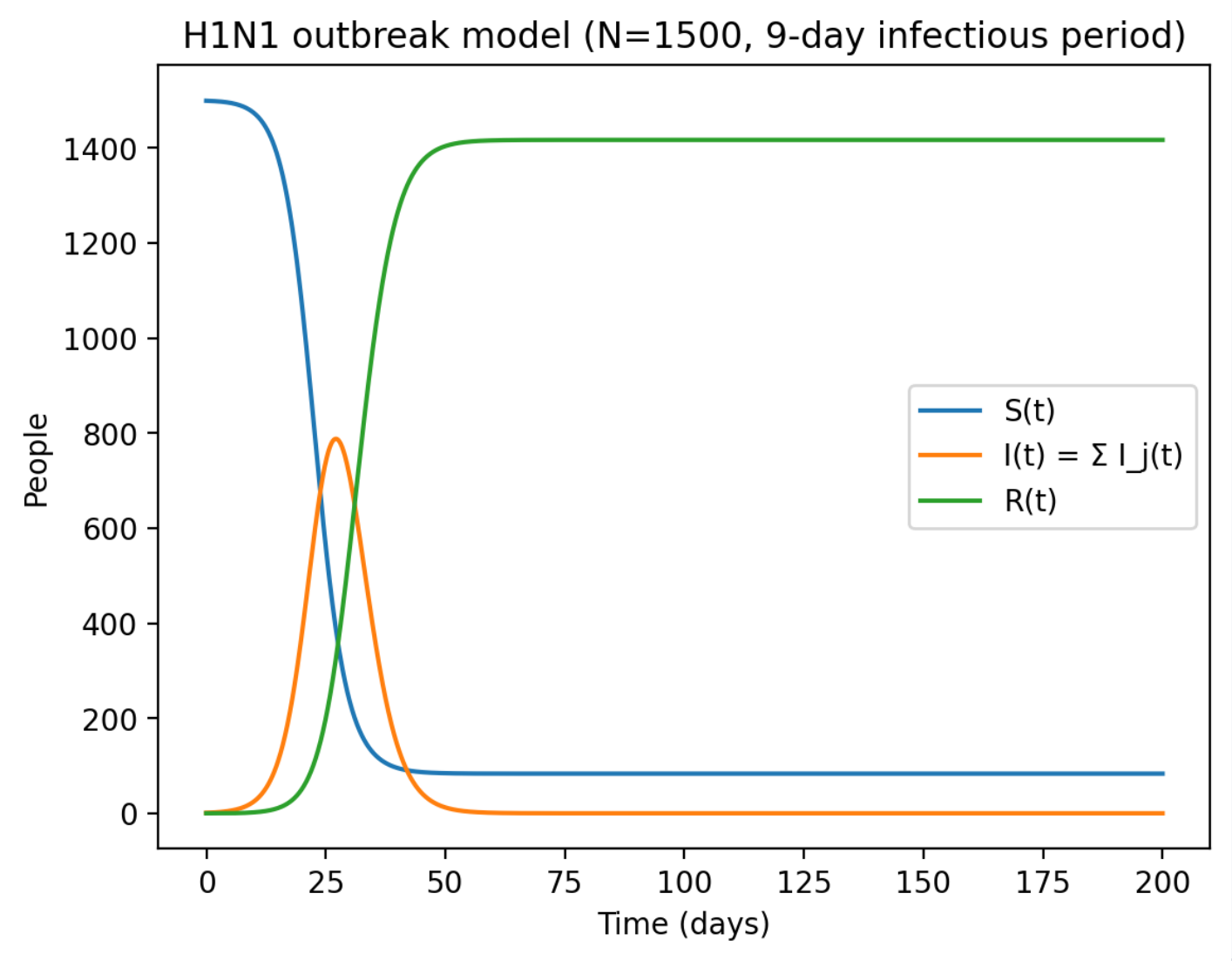
matplotlib

Python

```python
def save_plots(results):
    t = results["t"]
    S = results["S"]
    I_total = results["I_total"]
    R = results["R"]
    I_stages = results["I_stages"]

    # Graph 1: S(t), I(t), R(t)
    plt.figure(figsize=(8, 5))
    plt.plot(t, S, label="S(t)")
    plt.plot(t, I_total, label="I(t)")
    plt.plot(t, R, label="R(t)")
    plt.savefig("sir_total.png")
    plt.close()
```

Figure 1: S(t), I(t), and R(t)

Overall epidemic trajectory



How to read the graph

- S(t) starts near 1500 and drops quickly once transmission accelerates.
- I(t) rises sharply, peaks at about 788 people around day 27, then falls as the susceptible pool is depleted.
- R(t) increases steadily because recovered individuals accumulate and remain immune.

Interpretation

This is the classic epidemic wave: rapid growth, peak infection, then decline as susceptibles run out.

Graph 2 Code

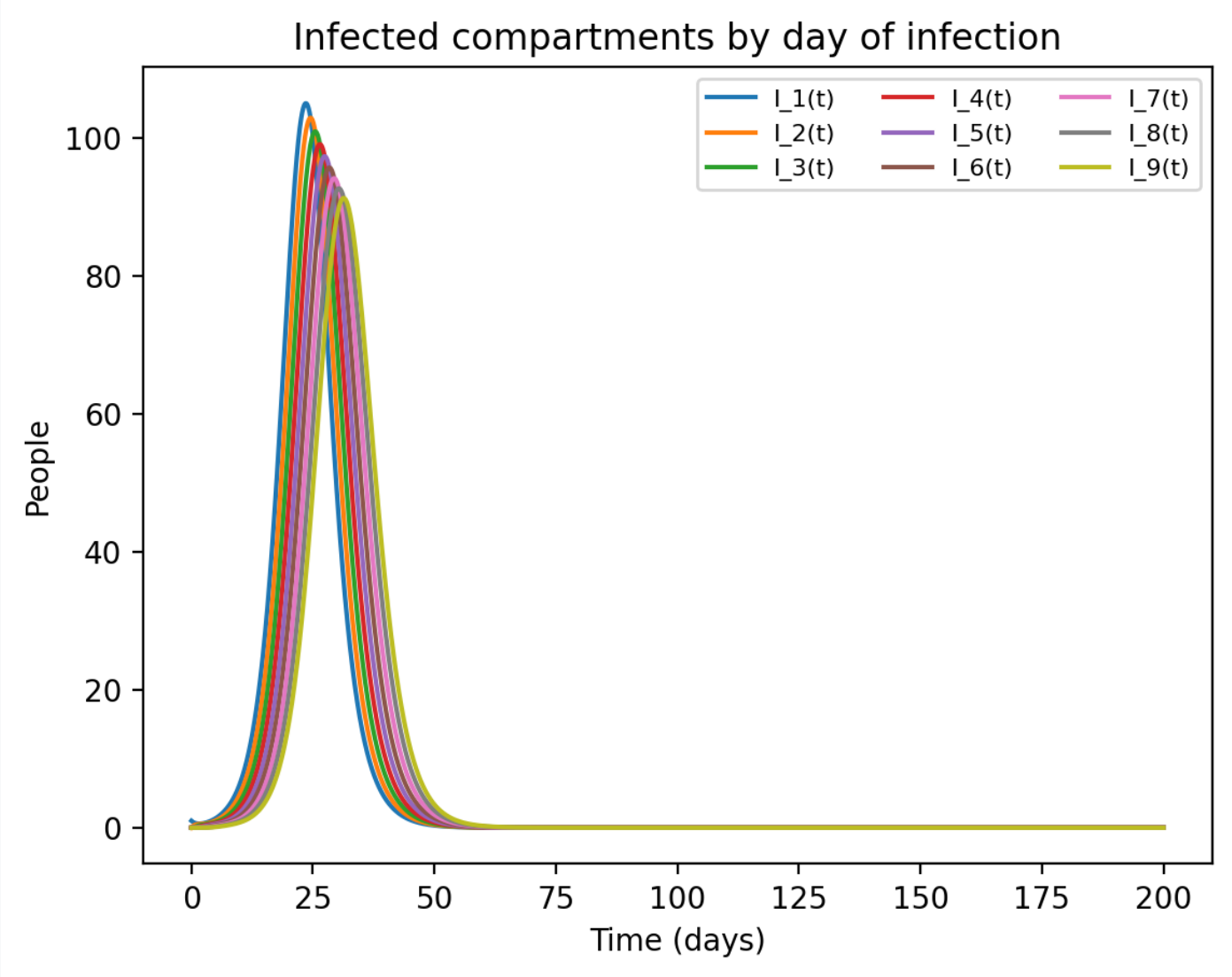
matplotlib

Python

```
# Graph 2: I1(t) to I9(t)
plt.figure(figsize=(8, 5))
for j in range(N_STAGES):
plt.plot(t, I_stages[j], label=f"I_{j+1}(t)")
plt.savefig("infected_stages.png")
plt.close()
```

Figure 2: infected compartments by day of infection

Stages I_1 through I_9



What the stage curves show

- Each curve represents one day of infection, from I_1 to I_9.
- The peaks occur one after another because people move sequentially through the infected stages.
- The similar bell-shaped curves confirm a smooth day-by-day progression before recovery.

Interpretation

The multi-stage model captures the fixed 9-day infectious period much more explicitly than a one-compartment infected class.

Challenges and limitations

Conceptual difficulties encountered during the project

Main challenges

- Interpreting the infected compartments correctly: at first, it was easy to confuse $I_1, \ldots, I_9$ with cumulative infections, when they actually represent people currently in days 1 through 9 of infection.
- Understanding numerical methods: since the system is nonlinear, we had to use RK45 and root finding instead of solving the equations by hand, so learning what those methods were doing took extra effort.
- Calibrating the transmission parameter β: it was challenging to understand how Brent's method searches for the correct β value and how that connects to the condition I(7)=10.

Conclusions

What this project shows

- An 11-compartment ODE model can represent a fixed 9-day infectious period in a biologically meaningful way.
- Calibrating β to satisfy $I(7) = 10$ gives $\beta \approx 0.339976$ and predicts a large outbreak in this closed population.
- The simulation peaks near day 27 and eventually leaves only about 83 people uninfected.
- Stage-structured models provide more detail about disease progression than a basic SIR formulation.

Bottom line

Mathematical modeling + numerical simulation turned the problem statement into concrete outbreak predictions.

Thank you

Questions?

References

Sources used in the report and presentation

[1] E. Sullivan, Numerical Methods: An Inquiry-Based Approach With Python, online textbook, Carroll College. Available online: https://numericalmethodssullivan.github.io/. Last updated: 2022-01-01.

[2] “SEIR model,” Wikipedia. https://en.wikipedia.org/wiki/Compartmental_models_in_epidemiology#The_SEIR_model.

[3] “Agent-based model,” Wikipedia. https://en.wikipedia.org/wiki/Agent-based_model.

[4] “Runge-Kutta methods,” Wikipedia. https://en.wikipedia.org/wiki/Runge%E2%80%93Kutta_methods.

[5] “Euler method,” Wikipedia. https://en.wikipedia.org/wiki/Euler_method.

[6] “Brent’s method,” Wikipedia. https://en.wikipedia.org/wiki/Brent%27s_method.

[7] SciPy Developers, “scipy.integrate.solve_ivp,” SciPy API Reference. https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html.

[8] SciPy Developers, “scipy.optimize.brentq,” SciPy API Reference. https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.brentq.html.

[9] Matplotlib Developers, “Matplotlib Documentation.” https://matplotlib.org/stable/contents.html